

Surrogate Model: GPR ve 5-fold Grouped CV

Ti-6Al-4V Çimentosuz Kalça İmplantı için Surrogate ML Modeli

Recep Kamil Fırat
Bilgisayar Mühendisliği · 2309011079

Mayıs 2026
ENS003 — Digital Twins for Health Sciences · Eğitmen: Prof. Dr. Şenol Pişkin

Bu sunumda

- Surrogate'ın amacı: FEA çıktısını milisaniyelere taşımak
- Neden Gaussian Process Regression (GPR), neden XGBoost değil
- Cross-validation stratejisi: *neden* 5-fold grouped
- 10 hedef için ölçtüğümüz R_{CV}^2
- Limitler ve dürüst raporlama

Sorumluluğum (yazılım ekibi)

Surrogate model seçimi, hyperparameter ayarı, cross-validation, `src/train.py` ve `models/`.

Problem

Mekanik ekibi ANSYS Static Structural ile her senaryoyu çözüyor:

- Tek bir çözüm $\sim 5\text{-}10$ dakika
- Bir kütle/K/açı kombinasyonunu kapsıyor
- İnteraktif dashboard için **uygun değil**

FEA çözümü	5–10 dk
Surrogate predict	≈ 1 ms
Hız çarpanı	$\sim 10^5$

Surrogate hedefi:

- Aynı girdiler için ~ 1 ms yanıt
- Eğitim grid'inin içinde **güvenilir interpolasyon**
- Posterior $\pm\sigma$ (kalibre belirsizlik)

Model seçimi — 270 satır, hedef SF_{neck}

Aynı veri seti üzerinde 5-fold grouped CV:

Model	R^2_{CV}	MAE_{CV}	Benzersiz çıktı / 51
XGBoost (depth=1, stumps)	0.986	0.032	18
XGBoost (depth=3)	0.988	0.024	27
Kernel Ridge (RBF)	-0.005	0.369	5
Gaussian Process (RBF + white)	0.9999	0.00077	51

Pürüzsüzlük testi: $K = 2.5, \theta = 0$ sabit, kütle slider'ı 51 örnekleme noktası boyunca taranıyor. GPR her noktada farklı bir değer üretiyor (sürekli eğri); ağaç tabanlı modeller az sayıda farklı değere düşüp **basamaklı** fonksiyon veriyor.

162 benzersiz noktada XGBoost yaprak başına ~ 6 örnek görüyor; bu da step-function davranışını üretiyor.

GPR mimari

```
from sklearn.gaussian_process import GaussianProcessRegressor
from sklearn.gaussian_process.kernels import RBF, ConstantKernel, WhiteKernel

kernel = (
    ConstantKernel(1.0, (1e-3, 1e3))
    * RBF(length_scale=1.0, length_scale_bounds=(1e-2, 1e3))
    + WhiteKernel(noise_level=1e-3, noise_level_bounds=(1e-6, 1e-1))
)

base = GaussianProcessRegressor(
    kernel=kernel, normalize_y=True,
    n_restarts_optimizer=5, random_state=42, alpha=1e-8,
)

model = TransformedTargetRegressor(
    regressor=base, transformer=StandardScaler(),
)
```

- Kernel hiperparametreleri marginal-likelihood ile fit
- `normalize_y` + `TransformedTargetRegressor` $\Rightarrow$ hedef ölçeği bağımsız
- Posterior kapalı form $\Rightarrow$ her tahminde σ bedava

CV stratejisi: *neden* grouped?

Problem: 270 satırın 108'i ANSYS re-run'ı — aynı (m, K, θ) noktasında ufak sayısal farklarla tekrar eden satırlar.

- Standart KFold / LOO kullanırsak: aynı grid noktası hem train hem test setine düşebilir $\Rightarrow$ *leakage*
- Görünür sonuç: R^2 yapay olarak ≈ 1.0 'a çıkar
- Gerçek out-of-sample performansını ölçemeyiz

Çözüm: GroupKFold($n_splits=5$), group ID Ata'nın grid_groups() fonksiyonundan geliyor:

"m | K | theta " $\Rightarrow$ re-run'lar aynı fold'da

Eğitim döngüsü

```
from sklearn.model_selection import GroupKFold

def _grouped_cv_eval(X, y, groups, n_splits):
    preds = np.full(len(y), np.nan)
    for tr, te in GroupKFold(n_splits).split(X, y, groups=groups):
        model, _ = _make_estimator()
        model.fit(X.iloc[tr], y.iloc[tr])
        preds[te] = model.predict(X.iloc[te])
    return {
        "r2_cv": r2_score(y, preds),
        "mae_cv": mean_absolute_error(y, preds),
    }
```

Her hedef için:

- ❶ 5 fold üzerinde out-of-fold tahmin
- ❷ Tüm satırlar üzerinde final fit $\Rightarrow$ `models/<target>.joblib`
- ❸ Metrikler `models/metrics.json`'a yazılır

CV sonuçları — 10 hedef

Hedef	R^2_{CV}	MAE _{CV}
safety_factor_neck_min	0.9999	7.7e-4
safety_factor_stem1_min	0.9999	8.3e-4
safety_factor_stem2_min	0.9997	4.4e-3
max_equivalent_vonmises_stress_Pa	0.9998	0.53 MPa
avg_equivalent_vonmises_stress_Pa	0.9998	10.6 kPa
min_equivalent_vonmises_stress_Pa	0.034	(FEA gürültü tabanı)
max_principal_stress_Pa	0.9998	0.57 MPa
avg_principal_stress_Pa	0.9998	5.0 kPa
min_principal_stress_Pa	0.9998	0.15 MPa
max_total_deformation_m	0.9998	7.4×10^{-7} m

9/10 hedef $R^2 > 0.999$; min_equivalent hedefi $\sim 10^{-10}$ mertebesinde FEA çözücüsünün gürültü tabanı — dürüstçe raporlanıyor, dashboard'da gizli.

Dürüst limitler

- **İnterpolasyon güvenilir, ekstrapolasyon değil.** $m < 50$ kg, $K > 6$, $\theta > 20^\circ$ için tahminler dikkatle kullanılmalı.
- **18-yüz fixed support sabit.** Farklı fiksasyon desenleri modellenmiyor; yeni FEA grid gerekir.
- $F_y = 0$ **tüm satırlarda.** Düzlem dışı yükleme yok; gerçek aktivite çoğunlukla düzlem-içi olduğundan kabul edilebilir ama tam değil.
- **Kernel hiperparametreleri bazı hedeflerde sınıra yakın (ConvergenceWarning).** R^2 'de marjinal etki, ileride bound'lar genişletilebilir.

Fallback zinciri: GPR $\rightarrow$ XGBoost $\rightarrow$ sklearn-GBR — sklearn'in GPR'si bir ortamda yoksa diğerleri devreye girer (test_make_estimator_falls_back_to_sklearn_gbr).

Demo

```
$ ./run.sh train
[...]
$ python -c "
import json
m = json.loads(open('models/metrics.json').read())
for r in m:
    print(f'{r[\"target\"]:42s}    R2_cv={r[\"r2_cv\"]:.4f}')
"
safety_factor_neck_min                R2_cv=0.9999
safety_factor_stem1_min                R2_cv=0.9999
safety_factor_stem2_min                R2_cv=0.9997
[...]
```

Test sonucu: 27 passed, %96 coverage — dashboard `models/`'i okuyup her predict'te 10 model üzerinden geçiyor.

Teşekkürler.

Sorular?

- `src/train.py` (GroupKFold + GPR + XGB fallback)
- `models/metrics.json` (canlı CV skorları)
- `tests/test_train_predict.py`
- Geliştirme logu: `docs/PROJECT_NOTES.md` Sprint 3–4